

NAME: Ayaan Shaikh
Mobile no : 8055733707
Internship : Data Analytics using Python
Submission date: 25/05/2026

Task 81 : Use Word Clouds for text data visualization.

Objective

The main objective of using word clouds is to visually represent text data in a way that highlights the most important or frequently used words.

```
# 81. Use Word Clouds for text data visualization
from wordcloud import WordCloud
import matplotlib.pyplot as plt
text = "data science data analysis machine learning data visualization"
wc=WordCloud(background_color='white').generate(text)
plt.imshow(wc)
plt.axis('off')
plt.show()
```

Output :



In this code, I **created a word cloud visualization** using Python to represent the frequency of words in a text dataset. First, I imported the WordCloud library and matplotlib for visualization. Then, I defined a text string containing words like *data*, *science*, *analysis*, *machine learning*, and *visualization*. Using `WordCloud(background_color='white').generate(text)`, I generated a word cloud where the size of each word depends on how often it appears in the text (for example, the word “data” appears multiple times, so it appears larger). After that, I used `plt.imshow(wc)` to display the word cloud as an image, removed axis labels with `plt.axis('off')` for a cleaner look, and finally displayed the result using `plt.show()`. Overall, this code helps me visually highlight the most important and frequent words in the given text data.

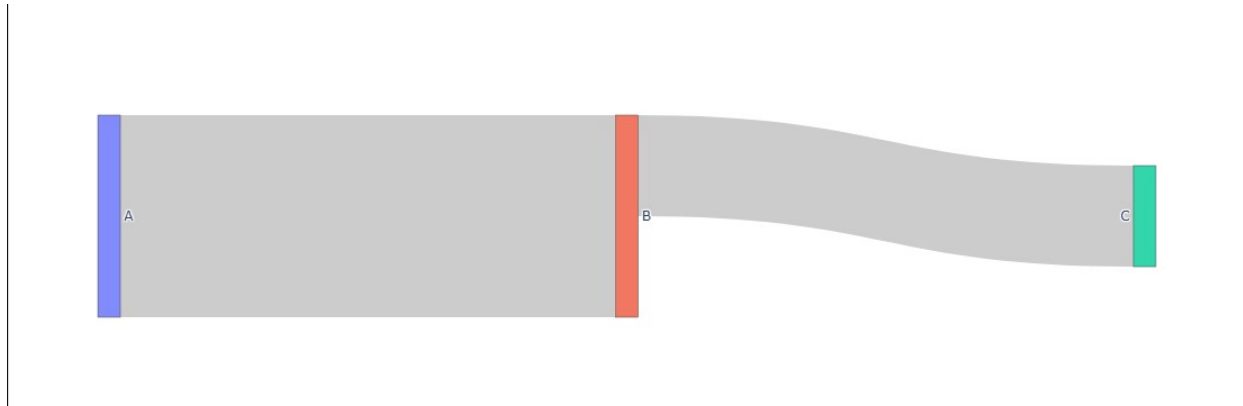
Task 82 : Use Sankey Diagram to show data flow .

Objective

The main objective of using a **Sankey Diagram** is to visually represent how data, resources, or values flow from one stage to another.

```
# 82. Use Sankey Diagram to show data flow
import plotly.graph_objects as go
fig = go.Figure(data=[go.Sankey(
    node=dict(label=["A", "B", "C"]),
    link=dict(source=[0,1], target=[1,2], value=[10,5])
)])
fig.show()
```

Output :



In this code, I created a Sankey diagram using Plotly to visualize the flow of data between different stages. First, I imported `plotly.graph_objects` to access advanced visualization features. Then, I defined three nodes labeled **A**, **B**, and **C**, which represent different stages in a process. Using the `link` parameter, I specified how data flows between these nodes: a flow of value **10** moves from **A** to **B**, and a flow of value **5** moves from **B** to **C**. The width of these flows in the diagram represents their quantity, making it easy to understand how much data is moving at each step. Finally, I used `fig.show()` to display the interactive Sankey diagram. Overall, this code helps me visually track how data flows and decreases across different stages.

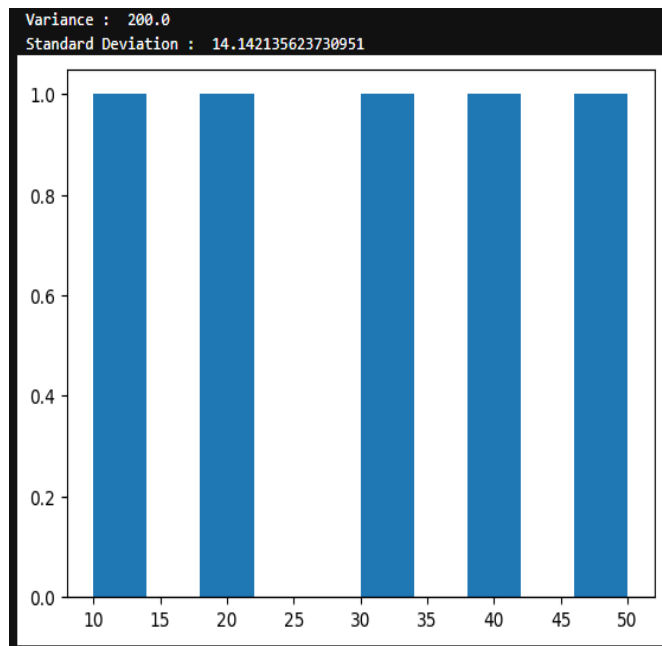
Task 83 : Create visuals for Variance & Standard Deviation.

Objective:

The main objective of creating visuals for **Variance** and **Standard Deviation** is to understand how data is spread or dispersed around the mean.

```
# 83. Create visuals for Variance & Standard Deviation
import numpy as np
data=[10,20,30,40,50]
print("Variance : ",np.var(data))
print("Standard Deviation : ",np.std(data))
plt.hist(data)
plt.show()
```

Output :



In this code, **I calculated and visualized the variability of a dataset using variance and standard deviation.** First, I imported the NumPy library and created a dataset [10, 20, 30, 40, 50]. Then, I used `np.var(data)` to calculate the **variance**, which tells me how spread out the values are from the mean, and `np.std(data)` to calculate the **standard deviation**, which gives the spread in the same unit as the data. After calculating these values, I used `plt.hist(data)` to create a **histogram**, which visually shows how the data is distributed. Finally, `plt.show()` displays the graph. Overall, this code helps me understand both numerically and visually how consistent or spread out my data is.

Task 84 : Understand impact after using visualization tools .

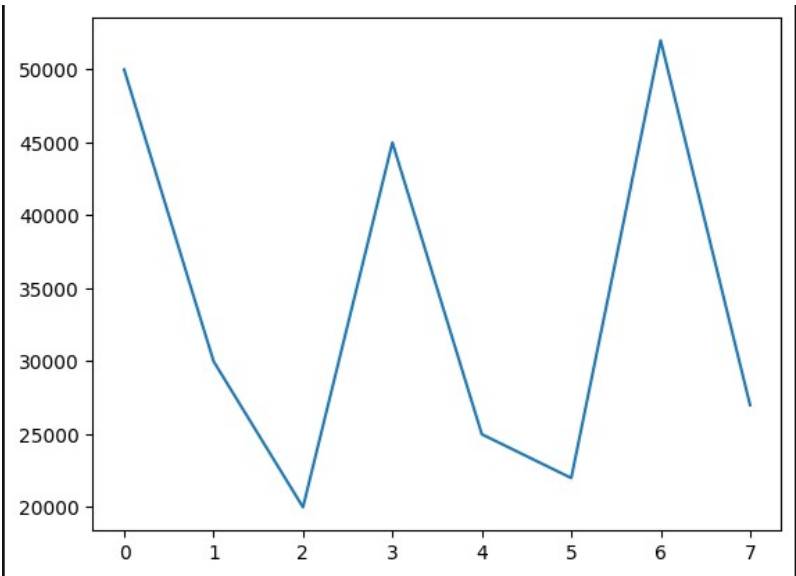
Objective

The main objective of using visualization tools (like word clouds, Sankey diagrams, histograms, etc.) is to **analyze how they improve understanding and decision-making from data.**

Before visualization :

	Region	Product	Sales	Quantity
0	East	Laptop	50000	5
1	West	Mobile	30000	10
2	North	Tablet	20000	7
3	South	Laptop	45000	4
4	East	Mobile	25000	8
5	West	Tablet	22000	6
6	North	Laptop	52000	5
7	South	Mobile	27000	9

After visualization :



After visualizing data, I can clearly understand patterns, trends, and relationships that are difficult to identify in raw data. Visualization transforms complex numbers and text into meaningful and easy-to-interpret insights. It helps me quickly detect outliers, compare values, and analyze the distribution of data.

As a result, decision-making becomes more accurate and efficient, and I can effectively communicate my findings to others. Overall, data visualization makes analysis faster, clearer, and more impactful.

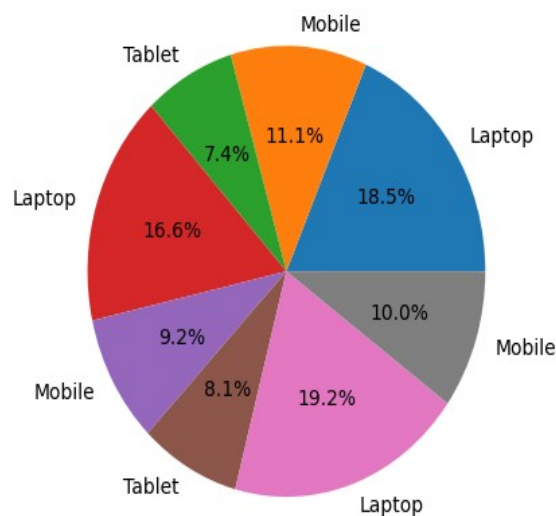
Task 85: Use Pie Chart when data proportion is needed .

Objective

A pie chart is best used when I want to show how different categories contribute to a whole (100%). It provides a clear and simple way to understand proportions at a glance.

```
#85. Use Pie Chart when data proportion is needed
import matplotlib.pyplot as plt
import pandas as pd
df=pd.read_excel("pivot_practice.xlsx")
df
labels=df['Product']
plt.pie(df['Sales'],labels=labels,autopct='%1.1f%%')
plt.show()
```

Output :



In this code, I created a pie chart to visualize the proportion of sales for different products. First, I imported the required libraries, matplotlib.pyplot and pandas. Then, I loaded the dataset from an Excel file (pivot_practice.xlsx) into a DataFrame. I selected the 'Product' column as labels and the 'Sales' column as values. Using plt.pie(), I plotted a pie chart where each slice represents a product's contribution to total sales. The parameter autopct='%1.1f%%' displays the percentage of each slice on the chart. Finally, I used plt.show() to display the visualization. Overall, this code helps me clearly understand how each product contributes to total sales in terms of proportion.

Task 86 : Use Colors to Highlight Trends .

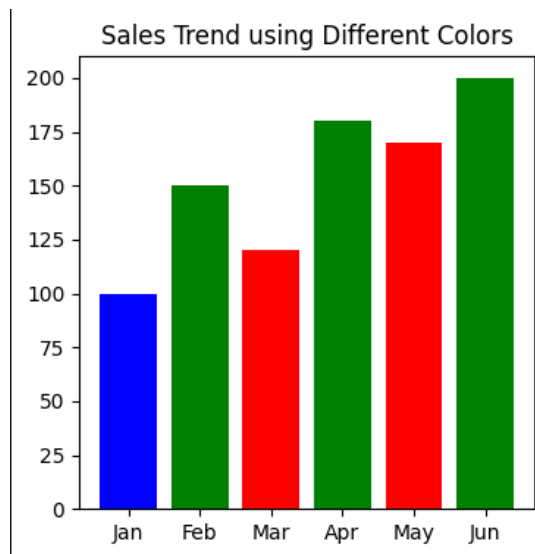
Objective

Using colors in charts helps you **quickly identify patterns, trends, and important insights** in your data. Instead of looking at plain numbers, colors make your visuals more intuitive and impactful.

```
# 86. Use colors to highlight trends
import pandas as pd
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [100, 150, 120, 180, 170, 200]
}
df=pd.DataFrame(data)

colors=[]
for i in range(len(df)):
    if i == 0:
        colors.append("Blue")
    elif df['Sales'][i] > df['Sales'][i-1]:
        colors.append("Green")
    else:
        colors.append("Red")
plt.figure(figsize=(4,4))
plt.bar(df['Month'],df['Sales'],color=colors)
plt.title("Sales Trend using Different Colors")
plt.show()
```

Output :



You created a data visualization that uses colors to highlight trends in sales over time. By comparing each month's sales with the previous month, you applied logic to identify whether the sales increased or decreased.

You then used different colors—blue for the starting point, green for an increase, and red for a decrease—to represent these changes in a bar chart using Matplotlib and pandas.

This approach makes it easy to understand patterns and trends at a glance without analyzing raw numbers, improving clarity and supporting better decision-making.

Task 87 : Use visual data for optimization (follow-up analysis) .

Objective

The objective of using visual data for optimization is to **analyze insights from charts and graphs to improve performance, identify problem areas, and make better data-driven decisions**. After creating visualizations, you examine patterns such as trends, peaks, declines, and outliers to understand what is working well and what needs improvement.

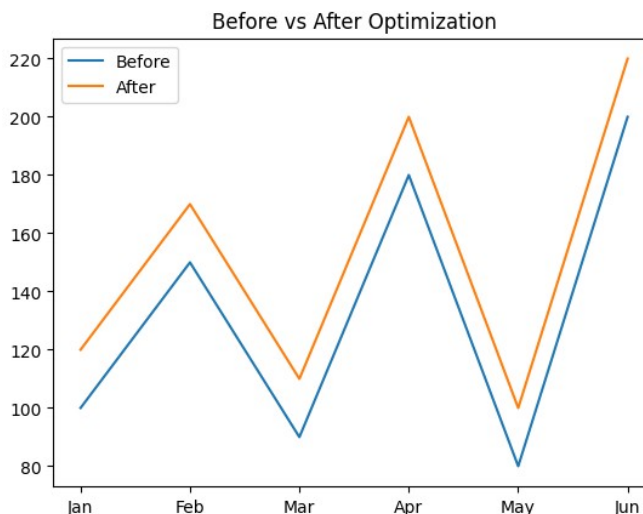

```
# 87. Use visual data for optimization (follow-up analysis)
import pandas as pd

data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [100, 150, 90, 180, 80, 200]
}

df = pd.DataFrame(data)
df['Improved_Sales'] = df['Sales'] + 20
plt.plot(df["Month"], df["Sales"], label="Before")
plt.plot(df["Month"], df["Improved_Sales"], label="After")

plt.legend()
plt.title("Before vs After Optimization")
plt.show()
```

Output:



I created a data visualization to perform follow-up analysis by comparing original sales with optimized sales. First, I built a dataset using pandas, then applied an optimization technique by adding a fixed value (+20) to generate an improved sales column. After that, I used Matplotlib to plot two line graphs—one representing the original sales (before optimization) and the other showing the improved sales (after optimization). This helped me visually analyze the impact of optimization, making it easier to identify improvements, compare performance across months, and support better data-driven decision-making.

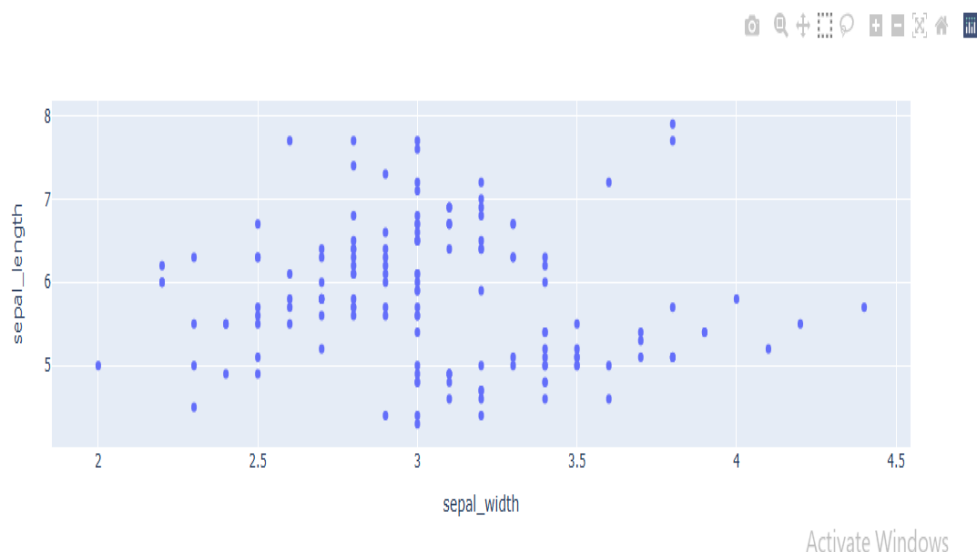
Task 88 : Create interactive visualizations (Plotly, Dash).

Objective

The objective of creating interactive visualizations is to allow users to explore data dynamically using tools like Plotly and Dash. These visualizations enable features like zooming, hovering, and filtering, making it easier to understand patterns and support better decision-making.

```
# 88.Create interactive visualizations (Plotly, Dash)
import plotly.express as px
df=px.data.iris()
fig=px.scatter(df,x='sepal_width',y='sepal_length')
fig.show()
```

Output :



I created an interactive scatter plot using Plotly. First, I loaded a sample dataset (iris dataset) using Plotly, then I visualized the relationship between sepal width and sepal length by plotting them on the x and y axes. Finally, I displayed the chart using `fig.show()`, which allows interaction like hovering, zooming, and exploring data points, making the visualization more insightful and user-friendly.

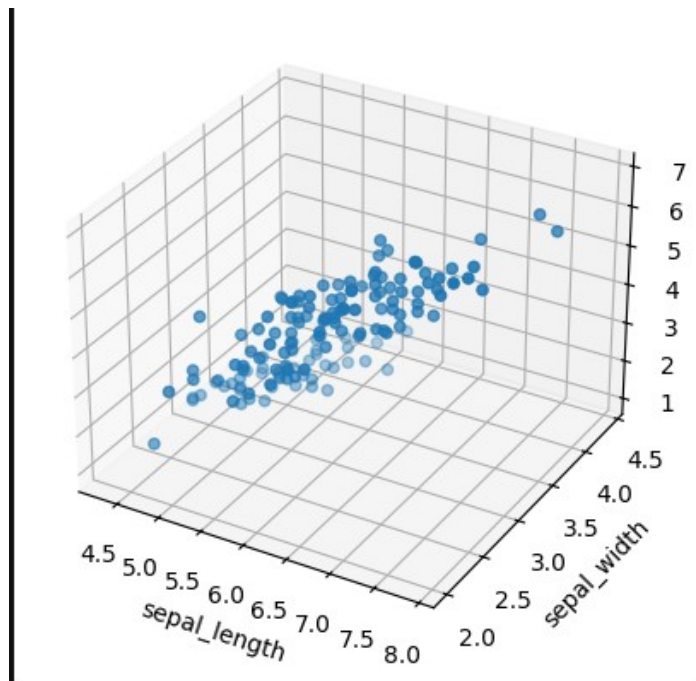
Task 90 : Visualize Multi-Dimensional data .

Objective

The objective of visualizing multi-dimensional data is to analyze multiple variables simultaneously to understand relationships, patterns, and trends in complex datasets. By using tools like Plotly or Matplotlib, different dimensions (such as size, color, shape, or position) can represent various data attributes in a single chart.

```
# Visualize Multi-Dimensional data
from mpl_toolkits.mplot3d import Axes3D
df=pd.read_csv("iris.csv")
fig=plt.figure()
ax=fig.add_subplot(111,projection='3d')
ax.scatter(df['sepal_length'],df['sepal_width'],df['petal_length'])
ax.set_xlabel('sepal_length')
ax.set_ylabel('sepal_width')
ax.set_zlabel('petal_length')
plt.show()
```

Output :



I visualized multi-dimensional data by creating a 3D scatter plot using Matplotlib. I loaded the iris dataset and plotted sepal length, sepal width, and petal length on the x, y, and z axes respectively.

This approach allowed me to represent three variables in a single visualization, making it easier to analyze relationships and observe how the variables interact with each other.

By using a 3D view, I was able to identify patterns, clusters, and possible correlations in the data more effectively than with a simple 2D chart, which helps in gaining deeper insights and improving data analysis.